

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 1(b)

Student Name: Appala Reddy

Roll No.: A24126552089

1. TITLE

Create Interactive Canvas Graphics and Design a Responsive Styled Webpage Using HTML5, JavaScript, and CSS3.

2. AIM

To create simple 2D graphics such as shapes, lines, and text using the HTML5 Canvas API with JavaScript, and to design a responsive webpage using CSS3 features including gradients, shadows, Flexbox, rounded corners, and hover effects.

3. OBJECTIVE

- To understand the purpose and working of the HTML5 <canvas> element.
- To obtain a 2D drawing context using `getContext("2d")`.
- To draw rectangles and circles using Canvas drawing methods.
- To create straight lines using `moveTo()`, `lineTo()`, and `stroke()`.
- To display custom text on a canvas using `fillText()`.
- To apply CSS3 gradients and text shadows to a webpage header.
- To create a navigation bar with interactive hover effects.
- To arrange webpage content using Flexbox.
- To use `border-radius`, `box-shadow`, and transform properties for visual styling.
- To create responsive cards and buttons with clean spacing and alignment.

4. THEORY

The HTML5 <canvas> element provides a drawing surface on a webpage. The canvas itself does not draw graphics automatically; JavaScript uses a drawing context to create shapes, lines, and text. CSS3 controls the appearance and layout of the surrounding webpage.

The main Canvas APIs and CSS properties used in this experiment are:

- `getContext("2d")` obtains the two-dimensional drawing context.
- `fillStyle` specifies the colour used for filled shapes and text.
- `fillRect(x, y, w, h)` creates a filled rectangle.
- `beginPath()` and `arc()` are used together to create a circular path.
- `moveTo()` and `lineTo()` define a straight line, while `stroke()` renders it.
- `fillText(text, x, y)` places text on the canvas.
- `linear-gradient()` creates a smooth colour transition in the page header.
- `:hover` changes the appearance of links, buttons, or cards when the pointer is over them.
- `box-shadow` and `border-radius` create depth and rounded visual elements.
- `transform: translateY()` moves a card upward during a hover interaction.
- `display: flex` and `gap` arrange cards in a flexible row with consistent spacing.

Basic flow: HTML loads the canvas → JavaScript obtains the 2D context → drawing commands create the graphics → CSS3 styles the header, navigation, cards, buttons, and footer.

5. ALGORITHM / PROCEDURE

1. Create an HTML file named `index.html` containing the canvas and webpage layout.
2. Add a canvas element with a fixed width and height.

3. Use `document.getElementById()` to access the canvas.
4. Call `getContext("2d")` to obtain the 2D drawing context.
5. Set a fill colour and draw a rectangle using `fillRect()`.
6. Create a circular path using `arc()` and fill the circle with a different colour.
7. Use `moveTo()`, `lineTo()`, and `stroke()` to draw a horizontal line.
8. Set a suitable font and use `fillText()` to display the name “Appala Reddy” on the canvas.
9. Create a header, navigation bar, three content cards, and a footer.
10. Apply a gradient background and text-shadow to the header.
11. Add hover effects to navigation links and buttons.
12. Use Flexbox to align the cards with suitable gaps and wrapping.
13. Apply rounded corners, shadows, and a small upward transform to cards on hover.
14. Save the file and open it in a browser.
15. Verify the canvas graphics, layout, hover effects, and footer display.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Appala Reddy - Styled Webpage</title>
<style>
.site-header {
  background: linear-gradient(to right, #30475e, #4aa3b8);
  color: white;
  font-family: 'Segoe UI', sans-serif;
  font-size: 30px;
  text-align: center;
  padding: 30px;
  text-shadow: 2px 2px 5px rgba(0,0,0,0.4);
}
.navbar {
  background-color: #182733;
  text-align: center;
  padding: 14px;
}
.navbar a {
  color: #f4f4f4;
  margin: 0 18px;
  text-decoration: none;
  font-family: Arial, sans-serif;
  font-size: 16px;
}
.navbar a:hover { color: #72d2e5; }
.section-title {
  text-align: center;
  font-family: 'Segoe UI', sans-serif;
  margin-top: 30px;
  color: #30475e;
}
.cards {
  display: flex;
  justify-content: center;
  gap: 25px;
  margin: 25px;
  flex-wrap: wrap;
}
.card {
  background-color: #ffffff;
  border: 2px solid #e0e0e0;
  border-radius: 14px;
  padding: 22px;
  width: 200px;
  box-shadow: 4px 4px 12px rgba(0,0,0,0.15);
  text-align: center;
  font-family: Georgia, serif;
}
.card:hover { transform: translateY(-5px); }
.card h3 { font-size: 19px; color: #30475e; }
.card p { font-size: 14px; color: #555; }
.card button {
  padding: 8px 18px;
  border: none;
  border-radius: 6px;
  background-color: #3d9bb2;
  color: white;
  cursor: pointer;
  margin-top: 12px;
  font-size: 14px;
}
.card button:hover { background-color: #256b7c; }
.site-footer {
  background-color: #182733;
  color: #ccc;
  text-align: center;
  padding: 18px;
  margin-top: 30px;
  font-family: Arial, sans-serif;
  font-size: 14px;
}
```

```

.canvas-section { text-align: center; margin: 25px 0; }
canvas { border: 1px solid #000; display: block; margin: 15px auto; }
</style>
</head>
<body>

<div class="canvas-section">
<h2 class="section-title">Canvas Graphics</h2>
<p>A simple rectangle, circle, line, and student name drawn using JavaScript on the HTML5 canvas.</p>
<canvas id="myCanvas" width="400" height="250"></canvas>
</div>

<script>
const canvas = document.getElementById("myCanvas");
const ctx = canvas.getContext("2d");

ctx.fillStyle = "#ef4b3a";
ctx.fillRect(20, 20, 100, 60);

ctx.beginPath();
ctx.arc(220, 60, 40, 0, Math.PI * 2);
ctx.fillStyle = "#3498db";
ctx.fill();

ctx.beginPath();
ctx.moveTo(20, 150);
ctx.lineTo(300, 150);
ctx.stroke();

ctx.font = "20px Arial";
ctx.fillStyle = "#30475e";
ctx.fillText("Appala Reddy", 20, 200);
</script>

<header class="site-header">
<h1>My Styled Webpage</h1>
<p>Showcasing modern CSS3 styling with gradients, shadows, Flexbox, and hover effects</p>
</header>

<nav class="navbar">
<a href="#">Home</a>
<a href="#">Canvas Art</a>
<a href="#">About</a>
</nav>

<h2 class="section-title">Web Development Highlights</h2>
<div class="cards">
<div class="card">
<h3>HTML5 Canvas</h3>
<p>Dynamic 2D rendering area displaying simple shapes and text using JavaScript.</p>
<button>Explore Graphics</button>
</div>
<div class="card">
<h3>Modern Styling</h3>
<p>Uses CSS gradients, custom shadows, smooth hover transitions, and rounded corners.</p>
<button>Learn More</button>
</div>
<div class="card">
<h3>Responsive Design</h3>
<p>Built with Flexbox, precise spacing, and flexible wrapping for different screen sizes.</p>
<button>View Details</button>
</div>
</div>

<footer class="site-footer">
<p>&copy; 2026 My Styled Webpage | Designed with HTML5 &amp; CSS3 by Appala Reddy</p>
</footer>

</body>
</html>

```

7. CODE EXPLANATION

Code / Property	Description
<canvas id="myCanvas">	Creates an empty drawing surface on which JavaScript renders graphics.
getContext("2d")	Returns the 2D drawing context containing Canvas drawing methods.
fillStyle	Sets the colour used by the next filled shape or text.
fillRect(x, y, w, h)	Draws a filled rectangle at the specified position and size.
beginPath(), arc(), fill()	Create a circular path and fill it with the selected colour.
moveTo(), lineTo(), stroke()	Define and render a straight line.
fillText(text, x, y)	Draws text on the canvas at the given coordinates.
linear-gradient()	Creates a smooth colour transition for the header background.
text-shadow	Adds a shadow effect behind header text.
:hover	Changes the appearance of links, buttons, or cards when the pointer is over them.
display: flex; gap	Arranges content cards in a flexible row with consistent spacing.
box-shadow; border-radius	Give cards depth and rounded corners.
transform: translateY(-5px)	Moves a card upward slightly when the user hovers over it.

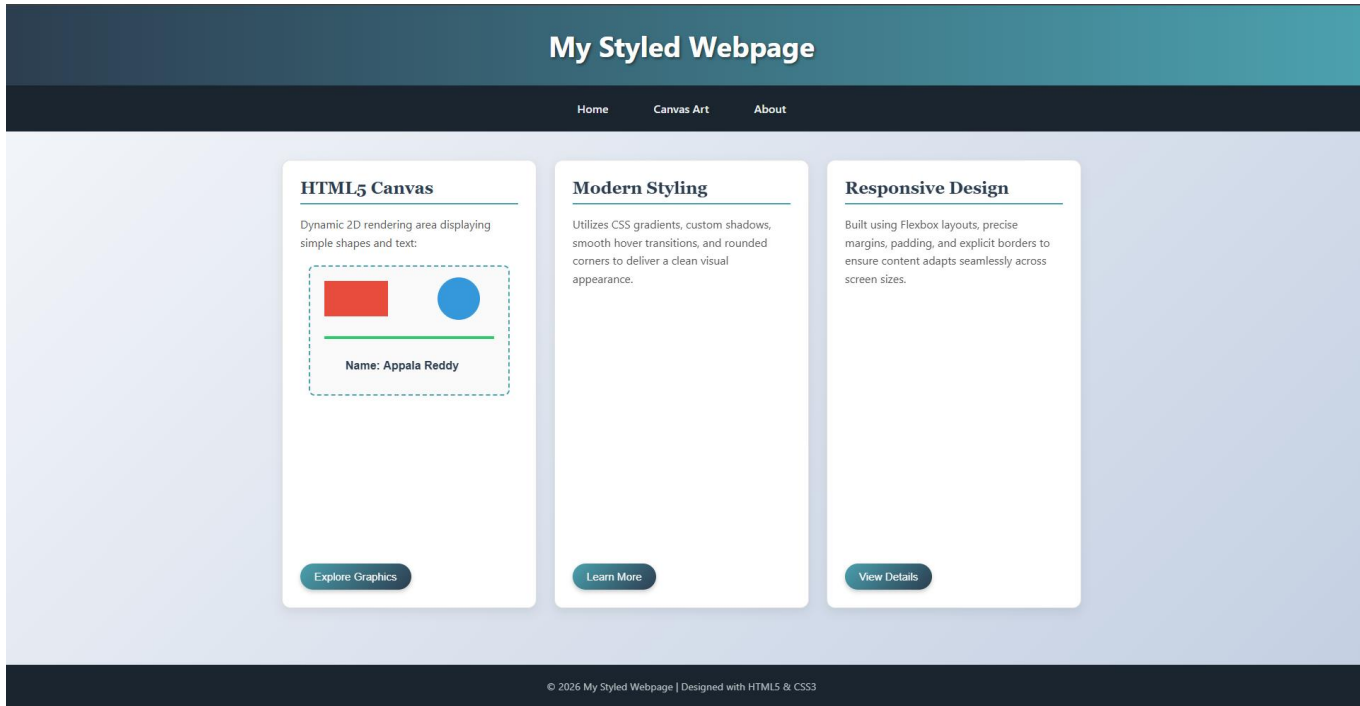
8. TEST CASES AND EXECUTION DATA

The webpage was opened in a browser and the canvas graphics, navigation, cards, buttons, hover effects, and footer were tested individually. The revised results are shown below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Canvas and styled webpage load correctly	Page loaded with all sections visible	Pass
TC-02	Check the rectangle	A filled rectangle appears on the canvas	Rectangle displayed correctly	Pass
TC-03	Check the circle	A coloured circle appears beside the rectangle	Circle displayed correctly	Pass
TC-04	Check the horizontal line	A straight line appears below the shapes	Line displayed correctly	Pass
TC-05	Check canvas text	"Appala Reddy" appears near the lower part of the canvas	Name displayed correctly	Pass
TC-06	Hover over a navigation link	Link appearance changes on hover	Hover effect displayed	Pass
TC-07	Hover over a card button	Button background changes to a darker shade	Button hover effect displayed	Pass
TC-08	Hover over a content card	Card moves slightly upward	Card lift effect displayed	Pass
TC-09	Check the three content cards	Cards appear in a row with even spacing	Cards arranged correctly using Flexbox	Pass
TC-10	Check footer	Footer displays copyright and student credit	Footer displayed correctly	Pass

9. OUTPUT

Output 1: Canvas Graphics and Styled Webpage



Output 2: The webpage demonstrates the HTML5 Canvas drawing, gradient header, navigation bar, responsive content cards, hover effects, and footer exactly as shown in the browser output above.

10. RESULT

Thus, interactive 2D graphics were successfully created using the HTML5 Canvas API and JavaScript, and a complete webpage was styled using CSS3. The experiment demonstrates Canvas drawing, gradients, shadows, Flexbox layout, rounded cards, navigation hover effects, button interactions, and responsive arrangement. All graphics and styled components were verified successfully in the browser.